

EWAT — Ontologie des Anomalies Kubernetes

Justification du périmètre retenu et anomalies omises

Document confidentiel — Stage de Recherche RCA / AIOps — Avril 2026

1. Contexte et contraintes expérimentales

Ce document justifie le choix des types d'anomalies retenus dans l'ontologie EWAT et explique pourquoi certaines catégories ont été écartées du périmètre expérimental. L'objectif n'est pas l'exhaustivité taxonomique, mais la validabilité expérimentale des hypothèses H1 (structurabilité), H2 (séparabilité du drift) et H3 (prédictibilité).

L'infrastructure expérimentale impose trois contraintes de sélection :

- Injectabilité contrôlée : chaque type doit être injectable de manière reproductible via Chaos Mesh, avec au moins 20–30 épisodes par type pour l'apprentissage contrastif (H1).
- Observabilité par la télémétrie disponible : le signal $S(t) \in \mathbb{R}^{N \times 13}$ (Prometheus, Jaeger, Loki) doit capturer une signature discriminante du type injecté.
- Faisabilité sur cluster 6 nœuds : certaines anomalies nécessitent une infrastructure multi-cluster, des workloads spécifiques ou des conditions impossibles à reproduire sur un cluster de développement.

2. Types d'anomalies retenus

L'ontologie réduite retient 5 modes de défaillance, 5 couches d'impact et 4 défaillances transverses, soit environ 35 types feuilles. Le tableau ci-dessous synthétise les modes retenus avec leur justification.

Catégorie	Types retenus	Injection Chaos Mesh	Signal télémétrique
Hard Failures	Crash, OOM, Network Partition, Image Pull, Node Down	PodChaos, NetworkChaos, native K8s	CPU, RAM, taux d'erreur, latence P99
Gray Failures	Fail-Slow, dégradation partielle, probe misconfig, diff. observabilité	StressChaos, NetworkChaos (delay)	Latence, spans anormaux, entropie logs
Systemic	Cascading, Metastable, Backpressure	Combinaison multi-faults + Locust	Propagation sur $G(t)$
Contention	CPU Starvation, Memory Pressure, I/O Throttling, Limplock, Noisy Neighbor, Resource Leak	StressChaos (CPU, mem, I/O)	CPU, RAM, saturation réseau
Config Failures	Traffic Misrouting, Bad Limits, Bad Selectors	Injection manuelle kubectl	Taux d'erreur, latence, fan-out

3. Anomalies omises et justification

3.1 Security & Identity Failures

Catégories omises : DDoS/Flood, Data Exfiltration, Privilege Escalation, Supply Chain Compromise.

Justification : Ces anomalies relèvent du domaine de la sécurité (SecOps) et non de la fiabilité (SRE/AIOps). Elles ne sont pas injectables via Chaos Mesh et nécessitent des outils spécifiques (pentesting, Falco). Leur signature télémétrique dans S(t) est faible ou absente : une exfiltration ne produit pas de signal discriminant sur les 13 dimensions retenues. La littérature [1][4] confirme que la détection d'anomalies de sécurité dans les microservices constitue un problème distinct.

3.2 Agentic AI Failures

Catégories omises : Cascade de corruption de données, épuisement d'inférence, Agent Timeout, Tool Loop.

Justification : Ces types supposent la présence de workloads agentic (agents LLM, pipelines d'inférence). Le banc d'essai EWAT utilise des microservices applicatifs classiques. Aucun workload agentic n'est déployé, rendant ces types non injectables et non observables. Ce domaine émergent ne dispose pas encore de taxonomie stabilisée.

3.3 Byzantine Failures

Catégories omises : Corruption d'état silencieuse, violation sémantique, Split Brain.

Justification : Les défaillances byzantines sont par définition difficiles à observer [12] : un composant produit des résultats incorrects sans signal d'erreur explicite. La télémétrie S(t) ne capture pas la sémantique des réponses applicatives. Chaos Mesh ne fournit pas de primitive d'injection byzantine. Cristian (1991) [12] définit ces défaillances comme nécessitant des mécanismes spécifiques (voting, consensus) absents du pipeline EWAT.

3.4 Data Plane Failures

Catégories omises : Cache stale, désynchronisation réplicas, schema migration failure.

Justification : Ces anomalies nécessitent un setup applicatif spécifique (Redis, PostgreSQL avec réplication). Elles dépendent de la couche applicative et non de l'infrastructure Kubernetes. Leur injection requiert des scripts ad hoc, non standardisables pour le protocole EWAT.

3.5 Temporal Failures

Catégories omises : Clock skew, dérive TTL certificats, race conditions sur finalizers, leader election race.

Justification : Le clock skew est difficile à injecter sans accès root aux nœuds (ntpd/chrony). Les race conditions sont des événements rares et non reproductibles de manière déterministe, violant le critère de reproductibilité (20+ épisodes par type).

3.6 Dependency Failures (mode)

Catégories omises : Circuit breaker mal configuré, timeout chain, retry budget, dependency version skew.

Justification : Partiellement capturées par les types retenus (retry storm en transverse, backpressure en systemic). En mode isolé, elles nécessitent une instrumentation fine du service mesh. Sur 6 nœuds avec une topologie limitée, le signal ne permet pas de les discriminer des gray failures.

3.7 Multi-Cluster / Edge

Catégories omises : Config drift, cross-cluster routing, CRD/version drift, partial propagation.

Justification : Le périmètre EWAT opère sur un cluster unique de 6 nœuds. Aucune fédération (KubeFed, Ligo, Submariner) n'est déployée. Ces anomalies sont structurellement inapplicables.

3.8 Control Plane

Catégories omises : etcd dégradation, API Server saturation, Scheduler failure, Controller Manager issues.

Justification : Leur injection risque de déstabiliser l'ensemble du cluster, empêchant la collecte de télémétrie. Sur un cluster de 6 nœuds, une panne etcd ou API Server provoque un arrêt complet, pas une dégradation progressive observable.

3.9 Secrets/Certs, Init/Sidecar, Jobs/CronJobs

Justification : L'expiration de certificats TLS est à cinétique lente (heures/jours), incompatible avec les fenêtres EWAT (minutes). Les init/sidecar crashes sont capturés indirectement par les Hard Failures. Les Job/CronJob failures nécessitent des workloads batch non déployés dans le banc d'essai.

4. Synthèse des critères de sélection

Catégorie omise	Injectable	Observable	Faisable 6N	Décision
Security & Identity	✗	✗	✓	Hors périmètre
Agentic AI	✗	✗	✗	Hors périmètre
Byzantine	✗	✗	✓	Hors périmètre
Data Plane	✗ (ad hoc)	~	✓	Hors périmètre
Temporal	✗	~	✓	Hors périmètre
Dependency (mode)	~	~	✓	Capturé indirectement
Multi-Cluster	✗	✗	✗	Inapplicable
Control Plane	✓	✗ (destructif)	✗	Risqué
Secrets/Certs	✓	✗ (lent)	✓	Cinétique incompatible

Init/Sidecar	✓	✓	✓	Capturé par Hard Failures
Jobs/CronJobs	✓	✓	✗ (no workload)	Hors setup

5. Références

5.1 Taxonomies et surveys d'anomalies

- [1] Soldani, J. & Brogi, A. (2022). Anomaly Detection and Failure Root Cause Analysis in (Micro) Service-Based Cloud Applications: A Survey. *ACM Computing Surveys*, 55(3), 1–39.
- [2] Chandola, V., Banerjee, A. & Kumar, V. (2009). Anomaly Detection: A Survey. *ACM Computing Surveys*, 41(3), 1–58.
- [3] De Camilli, M. et al. (2022). Towards a Fault Taxonomy for Microservices-Based Applications. *IEEE ICSA*.
- [4] A Comprehensive Taxonomy and Comparative Analysis of Fault Tolerance Mechanisms for Cloud-Native Microservices (2026). *IJISRT*, Vol. 11.
- [5] Fu, Y. et al. (2025). Intelligent Root Cause Analysis in Microservice Systems: A Survey. *ACM Computing Surveys*.
- [6] Zamanzadeh Darban, Z. et al. (2024). Deep Learning for Time Series Anomaly Detection: A Survey. *ACM Computing Surveys*.

5.2 Gray Failures et observabilité différentielle

- [7] Huang, P., Guo, C., Zhou, L., Lorch, J., Dang, Y., Chintalapati, M. & Yao, R. (2017). Gray Failure: The Achilles' Heel of Cloud-Scale Systems. *HotOS'17*. ACM.
- [8] Huang, P. et al. (2018). Capturing and Enhancing In Situ System Observability for Failure Detection. *OSDI'18, USENIX*.
- [9] GrayScope (2024). Non-Intrusive Gray Failure Localization. *ACM FSE'24*.

5.3 Metastable Failures

- [10] Bronson, N., Aghayev, A., Charapko, A. & Zhu, T. (2021). Metastable Failures in Distributed Systems. *HotOS'21*. ACM, pp. 221–227.
- [11] Huang, L. et al. (2022). Metastable Failures in the Wild. *OSDI'22, USENIX*, pp. 73–90.
- [11b] Isaacs, R. et al. (2025). Analyzing Metastable Failures. *HotOS'25*. ACM.

5.4 Défaillances byzantines et modèles de fautes

- [12] Cristian, F. (1991). Understanding Fault-Tolerant Distributed Systems. *Communications of the ACM*, 34(2), 56–78.
- [13] Burns, B., Grant, B., Oppenheimer, D., Brewer, E. & Wilkes, J. (2016). Borg, Omega, and Kubernetes. *Communications of the ACM*, 59(5), 50–57.

5.5 Concept drift

- [14] Myrtollari, K. et al. (2025). An Unsupervised Concept Drift-Aware Anomaly Detection System for Microservices on Kubernetes. *SSRN*.
- [15] Hinder, F. et al. (2024). Concept Drift in Unsupervised Data Streams. *Frontiers in AI*.

5.6 RCA et méthodes causales

- [16] Pham, T. et al. (2024). Root Cause Analysis for Microservices Based on Causal Inference: How Far Are We? ASE'24.
- [17] Chain-of-Event (2024). FSE'24.
- [18] RCACopilot (2024). EuroSys'24.
- [19] DynaCausal (2025). ICSE'25.

5.7 Détection d'anomalies K8s

- [20] Chen, Y. et al. (2022). Deep Attentive Anomaly Detection for Microservice Systems with Multimodal Time-Series Data. IEEE ICWS, pp. 373–378.
- [21] Enhancing Kubernetes Resilience through Anomaly Detection and Prediction (2025). arXiv:2503.14114.
- [22] Dependable Microservices in the Kubernetes era: A Practitioners Survey (2024). Journal of Internet Services and Applications.

5.8 Chaos Engineering

- [23] Chaos Mesh Documentation (2024). <https://chaos-mesh.org/docs/>
- [24] Heorhiadi, V. et al. (2016). Gremlin: Systematic Resilience Testing of Microservices. IEEE ICDCS, pp. 57–66.
- [25] Simonsson, J. et al. (2021). Observability and Chaos Engineering on System Calls for Containerized Applications. Future Generation Computer Systems, 122, 117–129.

5.9 SRE et résilience

- [26] Beyer, B., Jones, C., Petoff, J. & Murphy, N.R. (2016). Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media.
- [27] AWS (2024). Advanced Multi-AZ Resilience Patterns: Detecting and Mitigating Gray Failures. AWS Whitepaper.